

FidelityMCTS: Learning When to Think, Simulate, Verify, and Act

Anonymous Authors

Draft research proposal and system specification

July 2026

Abstract

Modern planners can query heterogeneous resources: small language models, learned world models, executable simulators, browser sandboxes, verifiers, and frontier models with controllable token budgets. Existing Monte Carlo tree search (MCTS) usually treats simulation as a homogeneous primitive, while model routers commonly choose one model for an entire query. We propose FIDELITYMCTS, a budgeted meta-planning framework that jointly searches over *task actions* and *compute actions*. At each branch, a compute policy chooses which simulator or model to query, how many tokens and rollout steps to allocate, whether to verify, and when to stop. Cheap biased predictions are paired with selectively acquired executable outcomes, enabling discrepancy learning, calibration, router improvement, and off-policy evaluation. FIDELITYMCTS is implemented as a research layer over MONTECARLOGYM, a modular, Gymnasium-compatible MCTS kernel that retains UCT, Bayesian posterior sampling, PUCT, advanced backups, RAVE, and MAST as composable presets. We specify the problem, architecture, learning loop, causal logging contract, and a preregisterable evaluation across synthetic, Gymnasium, executable tool, and browser environments. The central falsifiable hypothesis is that branch-level joint allocation improves the success–cost–risk Pareto frontier over single-model search, query-level routing, and fixed cascades. This draft reports the proposed method and experimental protocol; it deliberately does not claim unrun benchmark results.

Draft status. Architecture, APIs, and a controlled harness scaffold exist. Results sections are written as protocols and hypotheses. Numerical claims must be added only after the preregistered evaluation is completed.

1 Introduction

Test-time computation has become an important axis of capability. A system may sample several solutions, extend a reasoning trace, search a tree, call a stronger model, or execute a candidate in a sandbox. More computation can improve decisions, but its marginal value varies sharply across tasks and within a single search tree. Easy branches should not consume frontier-model tokens; consequential or ambiguous branches should not be trusted to a cheap simulator merely because it is fast.

MCTS offers a natural substrate for adaptive allocation. Its central purpose is to spend simulations non-uniformly across actions [2, 11]. Yet the simulation primitive is normally fixed: each rollout is treated as if it came from the same environment model at the same price. Neural MCTS changes the source of priors or values [18–20], but it does not usually make simulator identity, token budget, verification, and stopping a joint branch-level decision.

In parallel, LLM routing balances model quality and cost [14]. Recent work treats output length as a controllable variable [26], and metareasoning methods decide when intermediate reasoning is worthwhile [4, 13]. These methods motivate explicit compute control, but a query-level route cannot express the decision “use a cheap model on most branches, execute two disputed branches in a sandbox, then stop.” That distinction matters in sequential agent tasks.

The required resources are becoming practical. Language world models can simulate agentic transitions [10, 28], while code- and database-backed environments provide more expensive executable transitions and reliable verifiers [24]. BrowserGym and WorkArena expose realistic web and knowledge-work tasks through gym-like interfaces [5, 12]. The research opportunity is not merely to connect these components, but to learn how to allocate them during search.

We propose FIDELITYMCTS, whose object-level policy selects an environment action and whose meta-level policy selects a computation. A computation names a branch, a model or simulator, a token budget, a rollout horizon, and a verification choice. A stop action commits to the best current task action. The search is constrained by cost, token, latency, high-fidelity-call, and risk budgets.

The intended contributions are:

1. a formulation that separates task actions from branch-level compute actions and optimizes both under heterogeneous resource constraints;
2. a multi-fidelity MCTS architecture that combines cheap biased world models, executable simulators, foundation-model tiers, verification, and learned stopping without algorithm-specific engine forks;
3. a verified self-learning loop with discrepancy modeling, calibrated uncertainty, route-propensity logging, and optional causal/off-policy correction;

4. an open-source Gymnasium-compatible kernel and experiment contract spanning classical MCTS controls and agent-scale benchmarks.

The contribution is *not* the number of supported algorithms. Classical variants are necessary controls and reusable infrastructure. The scientific claim concerns adaptive allocation within the search tree.

2 Background and Related Work

2.1 Classical and Bayesian MCTS

UCT applies upper confidence bounds to tree selection [11]. MCTS has since accumulated selection, expansion, rollout, backup, parallelization, transposition, and domain-knowledge variants [2]. Crazy Stone introduced soft selectivity and studied mean, robust, and mixed backups [3]. Robust max uses the value of the most-visited move, avoiding a noisy maximum over empirical values.

Posterior-sampling approaches represent uncertainty directly. Bayesian reinforcement learning with MCTS can maintain Dirichlet transition beliefs [23]; posterior-sampling MCTS can use conjugate reward and transition models or root-sampled dynamics [1]. FIDELITYMCTS uses these mechanisms as possible uncertainty representations, but its compute policy also chooses *which evidence source* to sample.

2.2 Neural search and language-model planning

AlphaGo combines policy priors, value estimates, rollouts, and PUCT [19]; AlphaGo Zero removes rollouts and directly backs up a policy-value network [20]. MuZero plans with a learned latent dynamics model [18]. More recently, AlphaZero-like search has guided LLM decoding and supplied training targets [7]; MCTS has also been used to collect preference data for online optimization [25]. Tree-structured generative planning continues to improve with inference compute [27].

These results support search as a policy-improvement operator. Our focus is orthogonal: the simulation/evaluation backend is a portfolio, and allocation among portfolio members is itself planned.

2.3 Routing, metareasoning, and world models

RouteLLM learns cost-quality routing from preferences [14]. R2-Router jointly selects a model and output-length budget [26]. Rational metareasoning studies the value of computation (VOC), a long-standing limited-rationality concept [17], for selective LLM reasoning [4]. MetaBAMDPs extend metareasoning to uncertain environments [9]; recent metacognitive controllers choose operations such as expansion, pruning, repair, and stopping [13].

FIDELITYMCTS combines these ideas with MCTS at a different granularity. Routing can change between sibling branches and between depths, and it includes simulator fidelity and executable verification in addition to model/length.

Language world models now predict state transitions in agent environments [28]; WebDreamer uses an LLM as both world model and value function for web planning [10]. Agent World Model constructs 1,000 code- and database-backed executable environments [24]. This creates a useful asymmetry: learned models provide cheap speculative breadth, while executable environments provide costlier, higher-confidence evidence. C-MCTS also motivates low- and high-fidelity simulators for safe planning, though it uses the high-fidelity model offline to train a safety critic [15]. FIDELITYMCTS instead studies online, selective high-fidelity allocation.

2.4 Preference learning and causality

Search can optimize a reward or preference model and can generate targets for methods such as direct preference optimization [16, 25]. It does not eliminate the need to elicit human objectives. Indeed, stronger search can exploit reward-model errors more aggressively.

Selective routing creates observational data: expensive outcomes are observed where the current router chose to query. Causal routing addresses decision learning from such logged outcomes [22], while doubly robust estimators combine propensity and outcome models [6]. Causal world models may also improve long-horizon language-agent planning [8]. We adopt a narrow position: causal evaluation is valuable for routing logs and structured tool effects, but a universal causal world model is not a v1 prerequisite.

3 Problem Formulation

3.1 Object-level environment

Let an environment be a finite-horizon MDP or information-state process with states $s \in \mathcal{S}$, task actions $a \in \mathcal{A}(s)$, rewards r , and transition kernel P . In a partially observed setting, s denotes the planner’s information state, not inaccessible hidden state. A task policy π_A ultimately selects a real action for the live environment.

We assume a portfolio $\mathcal{M} = \{m_1, \dots, m_K\}$ of generative models. A member may be a learned transition/value model, LLM, reversible Gymnasium environment, executable sandbox, or verifier. Querying m at state-action pair (s, a) produces evidence

$$o \sim p_m(o \mid s, a, b, d),$$

where b is a token/sample budget and d is a rollout horizon. Evidence may contain a next state, reward, value estimate, uncertainty, terminal flags, and provenance. Each query

incurs a resource vector

$$g(o, m) = (c_{\text{money}}, c_{\text{token}}, c_{\text{time}}, c_{\text{accurate}}, c_{\text{risk}}).$$

No portfolio member is declared ground truth by type. “Accurate” is a relative fidelity label established empirically. Real outcomes and deterministic verifiers have distinct provenance.

3.2 Compute actions

At meta-history $h \in \mathcal{H}$, containing the partial search graph and all evidence, the compute policy π_C chooses

$$c = (s, a, m, b, d, v) \in \mathcal{C}(h),$$

where v indicates requested verification. It may instead choose STOP, after which $\pi_A(h)$ is executed.

The joint objective can be expressed as a constrained problem,

$$\max_{\pi_A, \pi_C} \mathbb{E}[G(\pi_A; h_{\text{stop}})] \quad (1)$$

$$\text{s.t. } \mathbb{E}\left[\sum_t g_j(c_t)\right] \leq B_j \quad \forall j, \quad (2)$$

$$\Pr(\text{unsafe action}) \leq \epsilon, \quad (3)$$

or as a Lagrangian utility

$$J = \mathbb{E}\left[G - \lambda^\top \sum_t g(c_t) - \kappa \text{Risk}\right].$$

We report Pareto frontiers because a single set of multipliers can conceal important quality-cost-risk trade-offs.

3.3 Discrepancy and value of computation

For a cheap model m and executable reference H , define paired discrepancy

$$\delta_m(s, a, h) = y_H(s, a) - \hat{y}_m(s, a, h).$$

A discrepancy model predicts a distribution over δ_m , rather than forcing cheap and executable estimates into one undifferentiated mean.

The ideal expected value of computation (EVC) is

$$\text{EVC}(c \mid h) = \mathbb{E}_{o_c}[U(\pi_A(h \cup o_c)) - U(\pi_A(h))] - \lambda^\top \mathbb{E}[g(c)]. \quad (4)$$

Exact EVC is generally intractable because it integrates future search changes. We study analytic approximations, contextual bandits, and learned EVC models.

4 FidelityMCTS

4.1 Search graph

We represent search as a state-action outcome graph. A state node stores state identity and expansion metadata. An action edge stores $N(s, a)$, $W(s, a)$, $Q(s, a)$, prior $P(s, a)$, optional Bayesian or AMAF statistics, and model-indexed evidence. Outcome links represent stochastic reward/next-state realizations. An iteration-local search path replaces a canonical parent pointer, allowing transpositions without ambiguous backup.

This representation is important for multi-fidelity evidence. Querying an accurate simulator does not overwrite a cheap prediction. Both remain attached to the edge with model version, uncertainty, and measured cost.

4.2 Two coupled policies

The tree policy selects a promising frontier under ordinary MCTS statistics. At that frontier, the compute policy repeatedly chooses evidence queries until a stop policy fires. The evaluator aggregates current evidence, and a backup operator propagates the resulting return. Thus UCT, PUCT, Thompson sampling, direct neural evaluation, rollouts, and alternative backups remain injected components rather than separate engine implementations.

Figure 1 shows the system. The core design constraint is that the live environment is never mutated by speculative search.

4.3 Routing and stopping

A reference score for a feasible compute action is

$$S_\theta(c, h) = \widehat{\Delta U}_\theta(c, h) - \lambda^\top \widehat{g}_\theta(c, h) - \kappa \widehat{\rho}_\theta(c, h), \quad (5)$$

where $\widehat{\Delta U}$ predicts decision improvement, $\widehat{g}$ resources, and $\widehat{\rho}$ risk. Hard budget masks are applied before scoring. The router chooses $\arg \max_c S_\theta(c, h)$ only when the maximum is positive; otherwise it stops.

Router features include search depth, value gaps, visits, predictive variance, model disagreement, expected cheap-to-accurate discrepancy, novelty, branch impact, reversibility, verifier availability, remaining budgets, and service load. The action may be factorized into branch, model, token budget, horizon, sample count, and verifier. This permits new portfolio members without retraining a flat classifier from scratch.

We include simple uncertainty-threshold and fixed-cascade policies as strong engineering baselines. A learned router earns its complexity only if it outperforms them under matched budgets.

4.4 Evidence aggregation and backup

An evidence aggregator produces a return distribution, not only a scalar. Candidate implementations are:

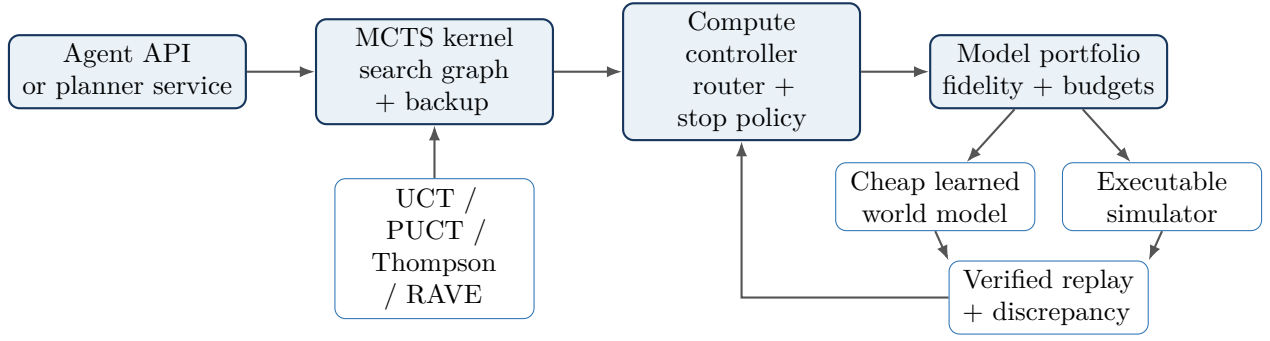


Figure 1: FIDELITYMCTS separates MCTS mechanics, algorithmic rules, and meta-level compute allocation. Cheap and executable evidence are paired in verified replay.

Algorithm 1: Adaptive multi-fidelity MCTS

1. Synchronize or create the root search graph.
2. While the resource ledger permits an iteration:
 - (a) select a path using the injected tree policy;
 - (b) expand a legal frontier;
 - (c) build a router context from evidence and budgets;
 - (d) query feasible compute actions until the router stops;
 - (e) aggregate evidence into an evaluation distribution;
 - (f) back up along the explicit iteration path;
 - (g) append a structured trace and measured usage.
3. Select a root task action under the configured decision rule.

Figure 2: Algorithm-name conditionals are unnecessary. AlphaGo Zero uses direct neural evaluation plus a no-rollout policy; UCT uses rollouts; adaptive variants insert model queries before evidence aggregation.

- verified-first selection;
- Bayesian fusion with model-specific likelihoods;
- cheap prediction plus discrepancy posterior;
- ensemble disagreement with conservative lower bounds;
- direct neural value or mixed value/rollout estimates.

The backup interface then chooses mean, distributional, robust, or mixed propagation. For Crazy Stone compatibility,

$$V_{\text{robust}}(s) = Q\left(s, \arg \max_a N(s, a)\right),$$

and

$$V_{\text{mix}}(s) = (1 - \alpha_s)V_{\text{mean}}(s) + \alpha_s V_{\text{robust}}(s),$$

where α_s can depend on visits and variance. Alternating-agent value perspective is configured explicitly.

4.5 Budgeted search procedure

The ledger reserves a conservative model quote before a query and commits measured usage afterward. A model

that exceeds its quote triggers a fault policy rather than silently breaking the search budget.

4.6 Verified self-improvement

Whenever executable or real evidence becomes available, the system stores

$$(h, s, a, m, \hat{y}_m, \hat{\sigma}_m, c, p(c | h), y_H, g, \text{versions}).$$

This supports four distinct objectives:

$$\mathcal{L}_{\text{value}} = \ell(\hat{y}_m, y_H), \quad (6)$$

$$\mathcal{L}_{\text{disc}} = \ell(\hat{\sigma}_m, y_H - \hat{y}_m), \quad (7)$$

$$\mathcal{L}_{\text{route}} = -\widehat{\text{EVC}}(c, h), \quad (8)$$

$$\mathcal{L}_{\text{cal}} = \ell_{\text{proper}}(\hat{p}_m, y_H). \quad (9)$$

Updates are staged: collect with frozen versions, validate, train candidates, evaluate offline, run randomized canaries, and promote only after quality, calibration, cost, and safety gates pass.

Search can also produce visit distributions or verified pairwise rankings for distillation and preference optimization [7, 25]. This can replace the policy-optimization step of some RLHF pipelines, but not human preference elicitation or reward validation. Model-only outcomes remain labeled synthetic.

4.7 Causal logging and evaluation

Because the router decides what to verify, verified replay is selected data. We log the probability of every chosen route and allocate bounded randomized audit traffic. For a deterministic target router π , a doubly robust estimate has the form

$$\begin{aligned} \hat{V}_{\text{DR}}(\pi) = & \frac{1}{n} \sum_{i=1}^n [\hat{r}(x_i, \pi(x_i)) \\ & + \frac{\mathbf{1}\{c_i = \pi(x_i)\}}{p_i} (r_i - \hat{r}(x_i, c_i))] . \end{aligned} \quad (10)$$

Overlap, stable treatment definitions, and sensitivity analyses are required. Causal claims are limited to interventions the system can define and support.

5 Software Architecture

5.1 MonteCarloGym kernel

MONTecARLOGYM provides a stable agent interface:

```
action = agent.compute_action(  
    sim_env, observation)
```

The user then calls the normal Gymnasium `env.step(action)` and passes the real transition to `agent.observe`. Gymnasium standardizes environment interaction [21], but generic generative-model protocols keep the kernel usable outside Gym.

The environment wrapper supports native snapshot/restore, clone state, safe deep copy, deterministic event reconstruction, or an external model. A transaction restores internal state, wrapper state, RNG streams, episode counters, and terminated/truncated flags even after exceptions. Environments that cannot satisfy this contract fail fast.

The search graph stores statistics on action edges and represents stochastic outcomes explicitly. Transposition tables use adapter-provided state keys. Tree reuse re-roots only after a matching real transition and invalidates incompatible model or state versions.

5.2 Dependency injection and presets

The engine depends on protocols for tree policy, expander, evaluator, rollout policy, backup, model portfolio, compute router, stop policy, reward source, state codec, and trace sink. Table 1 summarizes classical presets. They share the same engine.

5.3 Deployment and open-source surfaces

The dependency-light Python core is distributed through PyPI. A containerized planner service exposes HTTP/gRPC/MCP interfaces for enterprise and non-Python agents. Thin TypeScript adapters should expose an explicit `plan_and_execute` operation rather than assume a late tool hook can control earlier model selection. Router and discrepancy checkpoints, model cards, and permitted datasets can be released through model/data hubs.

Production deployments separate search coordinators, batched model gateways, simulator workers, replay storage, tracing, and asynchronous training. Every request has a budget envelope and idempotency key. Speculative execution uses a sandbox or dry run; irreversible or externally visible actions require an approval boundary.

6 Experimental Protocol

The empirical goal is to compare end-to-end planners under equal task distributions, model portfolios, external-action policies, and budgets. The task instance, not a tree simulation, is the independent unit.

6.1 Benchmark ladder

L0 tests identifiability, budgets, and routing invariants. L1 validates the classical kernel and stochastic state handling. L2 pairs a language world model such as Qwen-AgentWorld with executable Agent World Model environments [24, 28]. L3 uses BrowserGym and WorkArena [5, 12]. At least one L1 and one L2/L3 suite are required for a conference claim.

6.2 Baselines

Required comparisons are: direct policy/no search; cheap-only search; accurate-only search; fixed top- k cascade; random escalation matched to the method’s call rate; uncertainty threshold; query-level model router; query-level joint model/token router; FIDELITYMCTS; and, where computable, an oracle router. Classical UCT, PUCT, Thompson, robust/mix, RAVE, and MAST controls validate the implementation.

6.3 Metrics and primary analysis

We measure task success, return, regret, constraint satisfaction, tokens, normalized monetary cost, accurate calls, model calls by tier, environment steps, compute time, latency quantiles, memory, unsafe attempts, approval requests, and verifier pass rates. Diagnostics include value error, likelihood, interval coverage, calibration error, fidelity discrepancy, escalation precision/recall, stopping regret, and offline-evaluation bias.

The primary result is a success-cost-risk Pareto frontier across at least five budget levels. We report hypervolume under preregistered reference points, success at matched cost, cost at matched success, and area under the budget-performance curve. A single weighted aggregate is secondary.

6.4 Hypotheses and ablations

The confirmatory hypothesis is that FIDELITYMCTS improves Pareto hypervolume over single-model search and fixed/query-level routing. Secondary hypotheses test whether discrepancy-aware allocation improves over uncertainty alone, learned stopping is non-inferior in quality while reducing resources, verified replay improves held-out calibration, and causally corrected offline estimates better predict online router rankings.

Ablations remove discrepancy prediction, calibrated uncertainty, executable verification, token routing, variable depth, stopping, tree reuse, verified replay, propensity logging, causal correction, risk constraints, and transpositions. We separately test branch routing without model routing and model routing without branch routing.

6.5 Statistics and artifacts

Task instances and seeds are paired across methods. Synthetic suites use at least 30 independent seeds; expensive-

Table 1: Classical compatibility presets in the proposed open-source kernel.

Preset	Selection	Evaluation	Statistics	Backup/sharing
UCT	UCB1	random or heuristic rollout	edge N, W, Q	mean
DNG-MCTS / MCBRL	Thompson or root sampling	sampler re-ward/transition model	Normal-Gamma and Dirichlet	Bayesian update
Crazy Stone	soft/Boltzmann	rollout	visits, mean, variance	mean, robust, or mix
AlphaGo APV	PUCT	value network plus fast rollout	N, W, Q, P	mean, neural batching
AlphaGo Zero	PUCT	direct policy-value network	N, W, Q, P	mean, no rollout
RAVE / MAST	blended UCT/RAVE	globally biased rollout	direct, AMAF, global move values	direct+AMAF / base backup

Table 2: Planned benchmark ladder.

Level	Domain	Cheap	Accurate
L0	synthetic tree	biased oracle	truth oracle
L1	Gymnasium	learned dynamics	cloned env
L2	tool agents	language world model	code/DB env
L3	web/enterprise	LLM/DOM predictor	browser sandbox

suite sample sizes follow a preregistered pilot and power analysis. We use paired, task-stratified bootstrap intervals, report effect sizes, correct confirmatory comparison families, and use a non-inferiority margin for stopping claims. Splits are by task family when possible.

Each run releases its resolved configuration, source revision, environment and model versions, seeds, structured traces, per-episode JSONL, failures, and analysis. Provider non-determinism is measured by repeating a subset across time/load strata. The included L0 scaffold is a software smoke test, not a reported result.

7 Discussion

7.1 Why this may work

The method exploits three forms of heterogeneity. First, branch importance is unequal by construction in tree search. Second, model error is structured: cheap simulators are often adequate for routine transitions but fail on novel, ambiguous, or safety-critical states. Third, decision value is thresholded: an additional observation matters only when it can change the eventual action or reduce unacceptable risk. EVC gives a common language for these effects.

The architecture also makes enterprise scale plausible. Cheap local models can provide breadth, expensive services can be reserved for pivotal branches, and executable sandboxes provide evidence without touching production systems. Token efficiency becomes an outcome of the

compute policy rather than a fixed global limit.

7.2 Relationship to RLHF

Predictive search can serve as policy improvement at inference time or produce targets for distillation/preference optimization. In this limited sense it can replace “the RL step” of an alignment pipeline. It cannot determine what humans want, validate a reward model, or guarantee safety. Search may amplify reward hacking by finding adversarial high-score trajectories. We therefore retain reward provenance, component scores, uncertainty penalties, independent verifiers, and human approval for consequential actions.

7.3 Why causality is optional, not decorative

Causal correction addresses a concrete problem: the next router is trained and selected from outcomes acquired by the previous router. Randomized audits and propensity-based estimators can reduce selection bias. Structured causal world models may help in domains with stable, intervenable variables. However, general causal discovery from language traces is weakly identified; imposing a causal graph can add untestable assumptions and greatly expand scope. The core requires propensity logging and supports causal estimators, but does not label every predictive world model causal.

8 Limitations and Risks

EVC estimation. The value of a computation depends on counterfactual future search, making supervision sparse and policy-dependent. Simple threshold routers may remain competitive. Oracle-router experiments are needed to quantify available headroom.

High fidelity is not ground truth. Executable synthetic environments can omit production behavior. Browser sandboxes can differ from live sites. We must

report simulator-real discrepancy and use real outcomes only where safe and consented.

Distribution shift and feedback loops. Verified replay is concentrated near the router’s boundary. Without exploration and correction, the system may become confidently blind elsewhere. Jointly updating world model and router can destabilize targets.

Safety and security. Tree search can generate many tool actions, increasing prompt-injection and credential exposure. Speculation must be isolated; logs must redact secrets; side effects must be approval-gated. A sandbox reduces but does not eliminate risk.

Overhead. Maintaining uncertainty, discrepancy, traces, and routing can cost more than it saves on easy tasks. No-search and fixed-cascade baselines are therefore essential.

Research breadth. The full architecture spans MCTS, routing, world models, causal evaluation, and agent systems. A first paper should fix one strong simulator pair, one learned router family, and a limited set of classical controls rather than implement every advertised extension simultaneously.

9 Open-Source Plan and Broader Impact

The codebase will separate a minimal MONTECARLOGYM kernel from optional Gymnasium, PyTorch, Transformers, browser, and service extras. Source and issues live on GitHub; Python artifacts on PyPI; service images in an OCI registry; and permitted checkpoints/data with model cards on a model hub. Versioned research artifacts can receive archival DOIs.

Positive uses include lower-cost planning, reproducible MCTS research, safer sandbox-first agents, and better resource accounting. Negative uses include more capable automation of harmful tasks and reward-model exploitation. Adapters should default to read-only/dry-run operation, use scoped credentials, and expose both safety and cost telemetry. Training data and model-generated traces must be labeled by provenance.

10 Conclusion

FIDELITYMCTS reframes MCTS simulation as a portfolio allocation problem. The planner must choose not only what an agent should do, but how it should acquire enough evidence to decide: which branch to inspect, which model to trust, how many tokens and rollout steps to spend, whether to verify, and when to stop. This formulation connects classical and neural MCTS, LLM routing, world

models, metareasoning, preference optimization, and targeted causal evaluation while remaining testable through explicit budgets and Pareto analysis.

The proposed paper succeeds only if experiments show that joint branch-level control beats simpler routing and cascades on held-out executable tasks. The open-source architecture and harness are designed to make that claim reproducible—or falsifiable.

A API Contracts

Generative model. A model exposes stable identity, fidelity, a conservative pre-call resource quote, and an evaluation method. Its observation includes value, variance, measured cost, tokens, latency, risk, optional next state, terminal flags, and metadata.

Planner. The common planner receives a state, model portfolio, hard search budget, and seed. It returns a task action, predicted value, exact resource usage, and an auditable structured trace.

Reward source. Rules, environment rewards, preference models, humans, and verifiers implement one scoring protocol. A score carries scalar value, named components, verification status, source, and version.

Replay. A verified record pairs cheap predictions with executable or real outcomes and includes router propensity. Synthetic, executable, and real provenance are never collapsed.

B Implementation Invariants

1. Snapshot/restore reproduces environment and RNG state.
2. Planning cannot mutate the live environment.
3. Edge visits equal completed backups.
4. Resource limits are never exceeded, including failures.
5. Termination and truncation remain distinct.
6. Robust max selects the most-visited child.
7. Imaginary Bayesian transitions do not update the real root belief.
8. RAVE direct and AMAF updates are not double counted.
9. Accurate evidence augments rather than silently overwrites provenance.
10. Every learned route used for causal evaluation has a logged propensity.

C Submission Readiness Checklist

Before replacing this draft’s proposal language with empirical claims, require: one Gymnasium and one executable agent suite; all matched-budget baselines; at least five budget levels; held-out task-family evaluation; router and discrepancy ablations; calibration and safety reporting; paired uncertainty intervals; model/token/simulator cost accounting; and released configurations, code, and representative structured traces.

References

- [1] Aijun Bai, Feng Wu, and Xiaoping Chen. Posterior sampling for monte carlo planning under uncertainty. *Applied Intelligence*, 48:4998–5018, 2018. doi: 10.1007/s10489-018-1248-5.
- [2] Cameron B. Browne, Edward Powley, Daniel Whitehouse, Simon M. Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. A survey of monte carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1):1–43, 2012. doi: 10.1109/TCIAIG.2012.2186810.
- [3] Rémi Coulom. Efficient selectivity and backup operators in monte-carlo tree search. In *Computers and Games*, pages 72–83. Springer, 2006.
- [4] C. Nicolò De Sabbata, Theodore R. Sumers, Badr AlKhamissi, Antoine Bosselut, and Thomas L. Griffiths. Rational metareasoning for large language models. *arXiv preprint arXiv:2410.05563*, 2025.
- [5] Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, Léo Boissvert, Megh Thakkar, Quentin Cappart, David Vazquez, et al. WorkArena: How capable are web agents at solving common knowledge work tasks? In *International Conference on Machine Learning*, pages 11642–11662. PMLR, 2024.
- [6] Miroslav Dudík, John Langford, and Lihong Li. Doubly robust policy evaluation and learning. In *International Conference on Machine Learning*, pages 1097–1104, 2011.
- [7] Xidong Feng, Ziyu Wan, Muning Wen, Stephen Marcus McAleer, Ying Wen, Weinan Zhang, and Jun Wang. Alphazero-like tree-search can guide large language model decoding and training. *International Conference on Machine Learning*, 2024.
- [8] John Gkountouras, Matthias Lindemann, Phillip Lippe, Efstratios Gavves, and Ivan Titov. Language agents meet causality: Bridging LLMs and causal world models. *International Conference on Learning Representations*, 2025.
- [9] Prakhar Godara and Tilman Diego Alemán. Metareasoning in uncertain environments: A meta-BAMDP framework. *arXiv preprint arXiv:2408.01253*, 2026.
- [10] Yu Gu, Kai Zhang, Yuting Ning, Boyuan Zheng, Boyu Gou, Tianci Xue, Cheng Chang, Sanjari Srivastava, Yanan Xie, Peng Qi, et al. Is your LLM secretly a world model of the internet? model-based planning for web agents. *arXiv preprint arXiv:2411.06559*, 2025.
- [11] Levente Kocsis and Csaba Szepesvári. Bandit based monte-carlo planning. In *European Conference on Machine Learning*, pages 282–293. Springer, 2006.
- [12] Thibault Le Sellier De Chezelles, Maxime Gasse, Alexandre Drouin, Massimo Caccia, Léo Boissvert, Megh Thakkar, Tom Marty, Rim Assouel, Sahar Omid Shavegan, Lawrence Keunho Jang, et al. The BrowserGym ecosystem for web agent research. *arXiv preprint arXiv:2412.05467*, 2025.
- [13] Siyuan Ma, Bo Gao, Zikai Xiao, Hailong Wang, Xinlei Yu, Rui Qian, Jiayu Qian, Luqi Gong, and Yang Liu. CoT2-Meta: Budgeted metacognitive control for test-time reasoning. *arXiv preprint arXiv:2603.28135*, 2026.
- [14] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data. *International Conference on Learning Representations*, 2025.
- [15] Dinesh Parthasarathy, Georgios Kontes, Axel Plinge, and Christopher Mutschler. C-MCTS: Safe planning with monte carlo tree search. *Workshop on Safe and Trustworthy Agents at NeurIPS*, 2024.
- [16] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 36, 2024.
- [17] Stuart J. Russell and Eric Wefald. *Do the Right Thing: Studies in Limited Rationality*. MIT Press, 1991.
- [18] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, et al. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588:604–609, 2020. doi: 10.1038/s41586-020-03051-4.
- [19] David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *Nature*, 529:484–489, 2016. doi: 10.1038/nature16961.

- [20] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the game of go without human knowledge. *Nature*, 550:354–359, 2017. doi: 10.1038/nature24270.
- [21] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.
- [22] Asterios Tsourvas, Wei Sun, and Georgia Perakis. Causal LLM routing: End-to-end regret minimization from observational data. *arXiv preprint arXiv:2505.16037*, 2025.
- [23] Ngo Anh Vien, Wolfgang Ertel, Viet-Hung Dang, and TaeChoong Chung. Monte-carlo tree search for bayesian reinforcement learning. *Applied Intelligence*, 39:345–353, 2013. doi: 10.1007/s10489-012-0416-2.
- [24] Zhaoyang Wang, Canwen Xu, Boyi Liu, Yite Wang, Siwei Han, Zhewei Yao, Huaxiu Yao, and Yuxiong He. Agent world model: Infinity synthetic environments for agentic reinforcement learning. *arXiv preprint arXiv:2602.10090*, 2026. Accepted to ICML 2026.
- [25] Hang Xu, Zang Yu, Yehui Tang, Pengbo Hu, Yuhao Tang, and Hao Dong. MCTS-EP: Empowering embodied planning with online preference optimization. *arXiv preprint arXiv:2509.17116*, 2025.
- [26] Jiaqi Xue, Qian Lou, Jiarong Xing, and Heng Huang. R2-Router: A new paradigm for LLM routing with reasoning. *arXiv preprint arXiv:2602.02823*, 2026. Accepted to ICML 2026.
- [27] Jaesik Yoon, Hyeonseo Cho, Doojin Baek, Yoshua Bengio, and Sungjin Ahn. Monte carlo tree diffusion for system 2 planning. *International Conference on Machine Learning*, 2025.
- [28] Yuxin Zuo, Zikai Xiao, Li Sheng, Fei Huang, Jianhong Tu, Yuxuan Liu, Tianyi Tang, Xiaomeng Hu, Yang Su, Qingfeng Lan, et al. Qwen-AgentWorld: Language world models for general agents. *arXiv preprint arXiv:2606.24597*, 2026.